Design Patterns & Principles

1. What is a design pattern in Java? Why do we use it?

A **design pattern** is a proven, reusable solution to a common problem faced in software design. Think of it like a best-practice template or a recipe for solving a known issue. We use patterns because they speed up development and result in reliable, maintainable code that is easily understood by other developers.

2. What is the Singleton pattern? Give a simple real-life example.

The **Singleton pattern** ensures that a class can create only **one single object** throughout the entire application. We use it when we need exactly one manager for a crucial shared resource. A simple example is a **Logger** class that handles all application messages; you only want one logger writing to the file at a time.

3. What is the Factory pattern? When would you use it?

The **Factory pattern** is like a production line: you ask for a product (like "Car"), and it handles the complex creation details for you. You would use it when you have a general type but need to create different specific subtypes (e.g., creating a "CreditCardPayment" or "PayPalPayment" object). This keeps the main code clean by hiding the creation logic.

4. What is the Observer pattern? Explain in simple words.

The **Observer pattern** sets up a subscription system, like a magazine newsletter. You have one publisher object that automatically notifies all other subscribed listener objects whenever its state changes. It is useful for event handling, ensuring multiple parts of your application react instantly to a single update (e.g., pressing a button updates a graph and a status message).

5. What is the Strategy pattern? Give a small example.

The **Strategy pattern** lets you define different versions of an algorithm and swap between them easily at runtime. A small example is calculating a discount in an e-commerce store. You could have strategies for SeniorDiscount, StudentDiscount, or NoDiscount, and the system switches between them based on the user profile.

6. What is the Builder pattern? Why is it needed?

The **Builder pattern** helps create complex objects step-by-step, like ordering a custom pizza with specific toppings. It is needed because it solves the problem of having confusing constructors with many optional parameters. The Builder makes the object creation process much cleaner and less prone to errors.

7. What is SRP (Single Responsibility Principle)? One-line explanation.

The **Single Responsibility Principle** states that a class should have only one reason to change, meaning it should only have one job.

8. What is OCP (Open Closed Principle)? Basic definition.

The **Open Closed Principle** states that software entities should be **open for extension** (you can add new features) but **closed for modification** (you shouldn't have to change existing, working code).

Core Java & OOP Concepts

9. What is the difference between an Interface and an Abstract Class? (High level)

An **Interface** is a contract that only defines *what* a class must do (methods) but not *how*. An **Abstract Class** is a blueprint that can define both *what* a class does (abstract methods) and *how* it does some things (concrete methods and variables). A class can implement many interfaces but can only extend one abstract class.

10. What is Dependency Injection? Why do we use it?

Dependency Injection (DI) is a technique where a framework (like Spring) provides an object with the other objects it needs to work. We use it to make code modular and easy to test. By injecting dependencies, we can easily swap real components for fake testing components.

11. What is loose coupling and tight coupling?

Tight coupling means two classes are highly dependent on each other, so changing one forces you to change the other. **Loose coupling** means classes rely on interfaces or contracts,

allowing you to change the internal logic of one class without affecting the other. Loose coupling is always better for maintenance.

12. What is the difference between composition and inheritance?

Inheritance represents an **"is-a"** relationship (e.g., a Dog is an Animal). **Composition** represents a **"has-a"** relationship (e.g., a Car has an Engine). We prefer composition because it is more flexible and avoids the rigid structure that inheritance creates.

13. What is method overriding vs overloading?

Method Overloading is defining multiple methods in the **same class** with the same name but different input parameters. **Method Overriding** is when a **subclass** redefines a method that its parent class already provided.

14. What is immutability? Why is String immutable?

Immutability means that once an object is created, its state or value can never be changed. The **String** class is immutable primarily for security (like protecting network connections) and thread safety, ensuring its value remains consistent when shared between multiple threads.

15. What is the difference between HashMap and Hashtable? (Very common beginner question)

Hashtable is an older, legacy class that is **synchronized** (thread-safe), meaning it is slow because only one thread can access it at a time. **HashMap** is modern, **not synchronized** (fast), and is generally preferred unless you are working in a multi-threaded environment.